

Distributed Tracing for Cascading Changes of Objects in the Kubernetes Control Plane

Tomoyuki Ehira
ehira@inet.media.kyoto-u.ac.jp
Kyoto University
Kyoto, Japan

Daisuke Kotani
kotani@media.kyoto-u.ac.jp
Kyoto University
Kyoto, Japan

Yasuo Okabe
kotani@media.kyoto-u.ac.jp
Kyoto University
Kyoto, Japan

Abstract

Kubernetes is a container orchestration system that employs a declarative configuration management approach. In Kubernetes, each desired and actual state is represented in an “object”, and multiple controllers autonomously monitor related objects and update their objects towards the desired state in the control plane. Because of this design, changes to one object cause changes to other objects in a chain, and the time to complete these cascading changes is an important metric for cluster operators to maintain application service quality on the cluster. However, there is no practical way to observe this kind of cascading change in detail, such as sequence of objects updated and the elapsed time of this update. Distributed tracing techniques, which are used in the microservice architecture to monitor application performance, are mainly based on RPC model that clearly identify request and response pairs, thus they are not directly applicable to the control plane of Kubernetes, where controllers just monitor objects and autonomously trigger changes to objects. In this paper, we propose a system that automatically traces changes to objects in the control plane. Our method adds one identifier, a Change Propagation ID (CPID), to the metadata of each object, and the controller that observes an object change propagates CPID in the object to the objects that the controller updates. When multiple changes need to be merged on an object, a new CPID is generated, and a pair of the original CPIDs and the new CPID is sent to the trace server for recording and analysis. Through evaluation, we confirmed that the proposed system can capture change propagation and its time, and the overhead of the proposed system is not significant.

CCS Concepts

• **General and reference** → *Measurement*; • **Networks** → **Cloud computing**.

Keywords

Kubernetes, Control Plane, Tracing, Cascading Changes

ACM Reference Format:

Tomoyuki Ehira, Daisuke Kotani, and Yasuo Okabe. 2025. Distributed Tracing for Cascading Changes of Objects in the Kubernetes Control Plane. In *The 40th ACM/SIGAPP Symposium on Applied Computing (SAC '25)*,

March 31–April 4, 2025, Catania, Italy. ACM, New York, NY, USA, 8 pages.
<https://doi.org/10.1145/3672608.3707949>

1 Introduction

A declarative configuration management system defines and inputs a desired state and automatically updates configurations to match the actual state with the desired state. Kubernetes [16][19] is a de facto large scale container [1] management system that employs this approach.

In Kubernetes, operators configure resources and their objects by defining only the desired state. An “object” represents the desired and current state of each cluster function, and a resource is a set of objects of the same function. In the control plane of Kubernetes, controllers observe the current state of the objects that each controller is in charge of, and constantly perform a process (reconciliation loop) to get them into the desired state. This mechanism allows the controller to detect the mismatch between the current state and the desired state caused by failures or configuration changes, and autonomously fixes the current state toward the desired state. Since some controllers update their objects by observing the state of objects in other resources, changes occur like a chain among these controllers and objects until all related objects are updated.

One of the key metrics in a Kubernetes cluster is the time required to update all related objects when the desired state of one object is changed. For example, when requests to an application is suddenly increased, the operators will start more container replicas by updating the number of replicas to handle the requests, but the application cannot handle all the requests until all additional replicas are launched. Actually, there is a report [12] that the time to complete a particular process within the cascading changes is a bottleneck in the cluster operation. In order to identify where bottlenecks are, we need a mechanism to make cascading changes among related resources measurable to operators.

However, there is a challenge for observing these cascading changes. Because the updates of each object is processed autonomously in each controller and the operator does not specify the sequence of updating each object, it is not clear to which controller or object a change is propagated, thus it is hard to know or define when the change process for all related objects has completed. In addition, the objects to be affected by a change of a specific object depends on the system, such as the use of custom resources and plugins installed by the operators. Furthermore, the time to start and complete the update process of the objects depends on each controller, that is, the timing to reflect the changes to each object is not deterministic, and sometimes several changes are reflected together because the controllers only see the latest status of the objects.

There are several tools to measure the time of cascading changes, however, they cannot measure the cascading changes in general.



This work is licensed under a Creative Commons Attribution-NoDerivatives 4.0 International License.

SAC '25, March 31–April 4, 2025, Catania, Italy

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0629-5/25/03

<https://doi.org/10.1145/3672608.3707949>

K-Bench [20] and ClusterLoader [8] requires to define some types of changes to measure. Ehira et al. proposed a method [3] with manual logging by the operators, but it is costly because they require a deep knowledge of the controller design. The Kubernetes community has discussed extending the control plane's functionality to allow to measure cascading changes [11][10]. However, these are still in the discussion because of a problem in handling tracing information to be placed on the object including update and deletion. Distributed tracing techniques [13][4][2][7] are commonly used in the microservices architecture to monitor application performance, but they are not directly applicable to the control plane of Kubernetes; the microservices architecture is based on RPCs [18] and clearly identify the start and the completion of the requests, but in Kubernetes the controllers just monitor objects to know when to start processing, and never call functions on other controllers directly.

We propose a distributed tracing method in the Kubernetes control plane to facilitate observation of change propagation. We add change tracing identifiers to object metadata called Change Propagation ID (CPID). CPID is assigned to objects when processing objects in each controller, and is propagated to other objects that the controller updates. CPIDs are newly assigned when multiple changes are merged on an object, that is, the object is updated according to multiple objects having different CPIDs. All logs related to the handling of CPIDs are sent to the trace server for storing and analyzing logs to show the current status of the cascading changes. Our method implement the change propagation logic at the time of development of each controller, and the cluster operator can trace cascading changes of objects without deep understanding of each controller.

We confirmed that our system can easily trace change propagation and that the performance impact of the system on clusters is not significant.

The contributions of this paper are as follows.

- We summarize the challenges and requirements for tracing cascading changes specific to the Kubernetes control plane.
- This is the first distributed tracing system applied to the Kubernetes control plane, by the idea of combining updating CPIDs in the objects and analysing logs of how CPIDs are updated.
- We implement the proposed system and show that changes can be traced in the control plane and that the overhead is acceptable.

2 Related Work

2.1 Basics of The Kubernetes Control Plane

The control plane of Kubernetes declaratively manages containers and other computing resources for various functions. An object represents the desired and current state for each function in the control plane, and a resource is a set of objects with the same functionality. Each object or resource is managed by the corresponding controllers that observes the object's current state and updates it to the desired state. The following is a list of typical resources that also appear in this paper.

Deployment offers the functionality of gradually updating container images by creating ReplicaSets objects and changing the

number of replicas in multiple ReplicaSets with different container images.

ReplicaSet represents a set of the specified number of Pods of the same configuration, usually configured by Deployment controller, and ensures that the specified number of replicas is always available through creating and deleting Pod objects by ReplicaSet controller. **Pod** is a logical host running multiple containers that share storage and networking and controls the containers.

Service assigns IP addresses and DNS names to a set of Pods that matches a condition defined by the configuration of Service.

Endpoints holds information on IP addresses and port numbers of the Pods pointed to by the corresponding Service.

In addition, each host in the cluster runs kubelet, a component that manages computing resources on the host, and Scheduler in the control plane specify hosts to run Pods.

Figure 1 shows an example of cascading changes of Deployment, ReplicaSet, and Pod. When a Deployment object is created by the operator, the Deployment controller observes creation of the Deployment object and creates ReplicaSet objects, then the ReplicaSet controller observes the creation of the ReplicaSet objects and creates the Pod objects. Finally, each Pod is scheduled by Scheduler, and launched by kubelet. Records of objects are managed through API server in Kubernetes cluster.

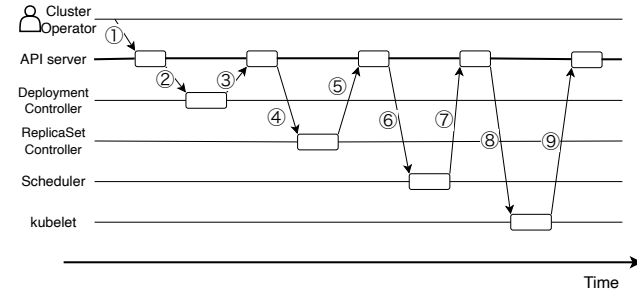


Figure 1: Example of cascading changes by Deployment, ReplicaSet, Scheduler, and kubelet. The numbers indicate the order of the changes.

Each cluster uses different sets of resources and objects. Operators will add more resources by defining custom resources, and some plugins like Cilium [14] and Istio [15] install custom resources managed by their own controllers. We need to support such flexibility in the control plane to trace cascading changes in general.

2.2 Measuring the Kubernetes Control Plane

K-Bench [20] and ClusterLoader [8] allow for measuring the time required for transitions between states of a pod object and the CPU usage during those transitions, mainly for operations on pods from Deployment. However, these supports only cases between some predefined resources, we cannot measure cascading changes involving other resources such as custom resources defined by operators. Ehira et al. [3] proposes a method assuming that the operator gives relationships in advance. However, proving relationships between objects in advance requires a signification effort for the operators.

Kubernetes community has been discussing ways to automatically trace change propagation by modifying the controllers and

other components of the control plane [11][10]. These proposes to use a distributed tracing technique to place trace context on the objects, similar to our work. However, there is a big concern that the size of the trace context will continue to grow because there is no method for deleting each trace context.

2.3 Distributed Tracing

Distributed tracing [13][4][2][7] is a technique for tracing how user requests are transmitted between services (applications running in containers) in a distributed system such as a microservice architecture. User requests are assigned a unique identifier (Trace ID), which is recorded with a timestamp when received by each service, and the same Trace ID is assigned when invoking other services. W3C Trace Context [21] and Zipkin B3 Headers [22] are widely used for the data format.

Information called a Span, which includes a Trace ID, Span ID, and timestamp when handling the request, is recorded by each service when it receives each request, and the Trace ID and its Span ID are propagated to other services when invoking other services. Functions for tracing requests such as assigning and propagating Trace IDs and Span IDs, are called instrumentation.

The motivation of Distributed Tracing is similar to our work, however, we cannot directly apply distributed tracing techniques to the Kubernetes control plane due to the difference of architectures between microservices and Kubernetes control plane. In microservices, services invokes other services explicitly because it employs RPC model, that is, relationship between services and timestamp to start and complete the process of each request can be clearly measurable. In Kubernetes control plane, each controller just see the current status of the objects and autonomously handle the change of the objects, so the controllers cannot see who and when the object is updated.

3 Design of the Proposed Method

3.1 Challenges

Distributed tracing in Kubernetes control plane has the following challenges and requirements.

The completion of the cascading changes is difficult to know. In the typical RPC model, the end of a series of requests can be defined as the point at which the response finally returns to the user. However, in the control plane, objects that are affected by a change of the specific object cannot be fully determined in advance due to autonomous design of the control plane. Even if we can determine the affected objects completely, it is difficult to know whether the change made by the controllers is caused by a particular change of the specific object, because the controllers see the latest status of the objects, not how the objects are updated.

For example in Figure 1, a change in the Deployment object is propagated to the ReplicaSet object and the Pod object. Other controllers will see the change of these objects and update the objects that they are in charge of, depending on the configuration of the control plane.

Multiple changes may be merged while changes are propagated. The controllers just see the latest status of the object. If the controller is overloaded and the processing is delayed, the controller will see the object that multiple changes are reflected. From the

perspective of tracing the cascading changes, this situation should be regarded that multiple changes are aggregated and merged, and that the controller handles multiple changes at once. So, we need to identify which changes are merged into the current object. This situation does not occur in the RPC model because requests and responses are explicitly mapped.

For example, when a cluster operator creates a Deployment and before the completion of this creation the automatic scaling function changes the number of Pods to be run in the cluster, there should be two changes, and the controllers see the latest number of Pods to be run in the cluster, not the number that the cluster operator first configured.

Avoid the excessive growth of the trace context As mentioned, it is hard to know when all changes is completed, so it is difficult to decide it is safe to delete the trace contexts on the objects. As a result, the trace contexts may be left on the objects for a long time, and the size of the trace contexts will grow when multiple changes are occurred. However, the size of the storage in the control plane is limited, so it is necessary to limit the size of the trace contexts in the objects regardless of a number of changes.

3.2 Overview

An overview of the proposed system is shown in Figure 2. To trace cascading changes, we assign one Change Propagation ID (CPID) and optionally ancestor CPIDs to each object. CPIDs are newly assigned from scratch, or based on multiple CPIDs when multiple changes are reflected in the object. The details of handling CPIDs are discussed in Section 3.3.1 and ancestor CPIDs in Section 3.5. Each controller sees the CPID when processing, and sends mergelogs and spans to the trace server when CPIDs on objects are changed. CPIDs will be also included in the normal logs. The trace server creates a merge graph from the received mergelogs, and responds to queries from operators that want to see the status of the cascading changes.

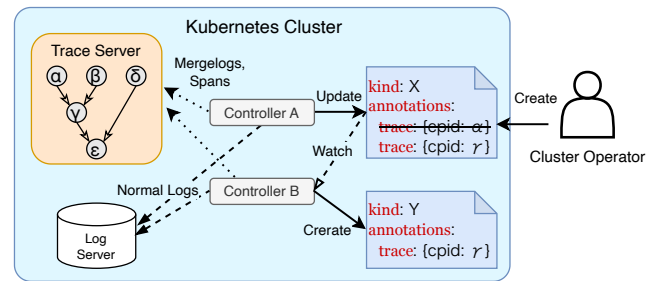


Figure 2: How the controllers work in the proposed method.

3.3 Components

3.3.1 Change Propagation ID (CPID). A Change Propagation ID (CPID) is an identifier for continuously tracing changes. In our method, only one CPID is carried per object, which is the entity for propagating changes. In our work, CPID is stored in the annotation field of the metadata in each object.

The basic lifecycle of a CPID is as follows. When a controller creates an object from scratch, which means without seeing any

objects, it assigned a new CPID and attach the CPID to the object. When a controller creates or updates an object as a result of seeing existing objects including the same object that the controller is updating, the controller assigns a new CPID and associates it with the CPIDs of the objects that the controller sees, i.e. the objects that causes the change. The rationale of this is to record which status of the objects the controller sees to update the object. Note that the controller sends the association of CPIDs to the trace server as mergelogs for analysis by the operators.

To eliminate performance bottlenecks and single points of failure by assigning CPIDs, identifiers such as UUID [9] that do not conflict even if they are generated independently in a distributed environment.

3.3.2 Mergelogs and Merge Graph. A mergelog is information that links a newly assigned CPIDs and the CPID on objects that are seen during the updates of the object where the new CPID is assigned, and sent by each controller to the trace server. The trace server constructs a merge graph that represents the relationship of CPIDs from the mergelogs, responds to queries about specific CPIDs such as related CPIDs that associates the queried CPIDs.

The fields in the mergelog are listed in Table 1. New CPID is a CPID that is newly generated, and the Source CPIDs are a list of CPIDs of the objects that the controller sees to update the object where the new CPID is assigned. The Timestamp field is used to remove old mergelogs and CPIDs from the merge graph.

Table 1: Fields of a mergelog

New CPID	A newly generated CPID
Source CPIDs	A list of CPIDs of the objects that the controller sees to update the object where New CPID is assigned.
Timestamp	Timestamp of mergelog creation

The trace server can simply creates the merge graph by each CPID as nodes and edges from New CPID to one of CPID in Source CPID for all Source CPIDs in the mergelogs. The merge graph becomes an acyclic directed graph because CPIDs are not reverted to the previous ones.

Deletion of nodes and edges from the merge graph can be done from the nodes that have no incoming edges, i.e., the nodes that are not merged into other CPIDs. When either side of the node on an edge is deleted, the edge is also deleted.

3.3.3 Spans. Span is responsible for recording the start and end times of the process as well as events during the process, similar to the distributed tracing in general. The actual implementation of the controllers are loops and each iteration of the loop consists of observing then updating objects and other computing resources, so, spans corresponding to each iteration is useful to identify bottlenecks in the cascading changes.

Table 2 shows the fields of the span in the proposed method. CPID refers to the CPID of the change the span is linked to. The process's start and end times are also recorded. Service field indicating which controller the span is, and the span's identifier, the Span ID, is also included.

The Parent ID slightly differs from the parent span in the distributed tracing in general. The first span created in each controller

is regarded as the root span (a span for which no parent span is specified). When an object change is observed, the span starts with the object's CPID, and the Parent ID field is left empty. When further changes to objects are made in the same controller during the span, the child span's identifier is generated, and the parent's Span ID is stored in the child span's Parent ID field.

Table 2: Fields of a span

CPID	CPID of the change that caused the process
Start Time	The timestamp when the process started
End Time	The timestamp when the process ended
Service	The name of the controller where the process occurred
Span ID	ID of this span
Parent ID	ID of the parent span

3.3.4 Trace Server. The trace server stores mergelogs and spans sent by controllers, construct merge graphs, and responds to queries from operators.

3.4 Example of CPID Handling, Mergelog and Merge Graph

Figure 3 shows how trace context including CPID is propagated through the objects in this system. The right side of the dotted line shows the mergelog sent to the trace server and the merge graph.

First, the cluster operator creates the Deployment object using, kubectl, a command line tool for operators. kubectl creates CPID α . There is no change to merge it for creation, so α is copied in the Deployment object. When creating ReplicaSet and Pod objects, CPID α is also copied. The controllers send a mergelog to the trace server indicating the new CPID α has been assigned, and a merge graph consisting only of α .

Second, the cluster operator creates a Service object. kubectl creates the CPID β and β is copied to the Service object. Similarly, when the Service controller creates the Endpoints object, it copies the CPID β . A mergelog is sent to the trace server indicating that a new CPID β has been assigned, and β is added to the merge graph.

Finally, the Endpoint controller observes the state of the Pod objects. It updates the Endpoints object based on the state of the Pod objects and existing Endpoint object, where the changes in α and β are reflected. The Endpoints controller assigns a new CPID γ and sends a mergelog $\{\alpha, \beta\} \rightarrow \gamma$ to the trace server, linking the Source CPIDs α and β with CPID γ . The trace server adds γ and edges from α to γ and β to γ .

3.5 Ancestor CPIDs

In the above method, if an object is changed and then the same object that is not updated by other factors is changed again, many unnecessary merging processes occur in the object where the changes are propagated, which are not necessary for change tracing. This causes undesirable effects, such as increased processing time for the cascading changes and increased size of mergelogs and merge graphs. For example in Figure 4, since γ reflects the changes of α , replacing α with γ is enough. However, each controller does not know that γ have the information of α , and unnecessary merge processings happen.

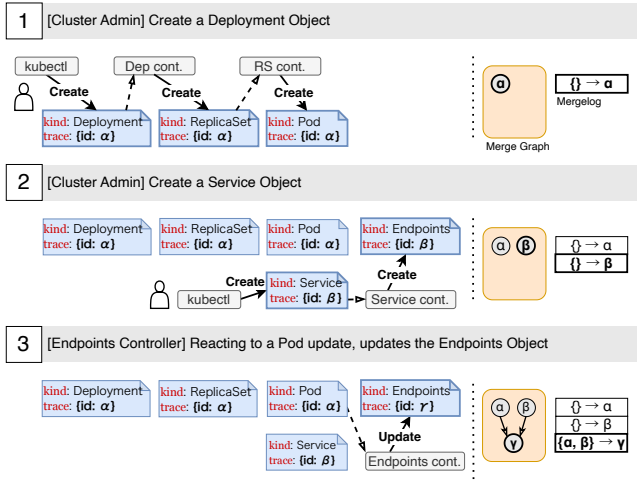


Figure 3: Proposed system propagating CPIDs and mergelog and merging graphs at each moment

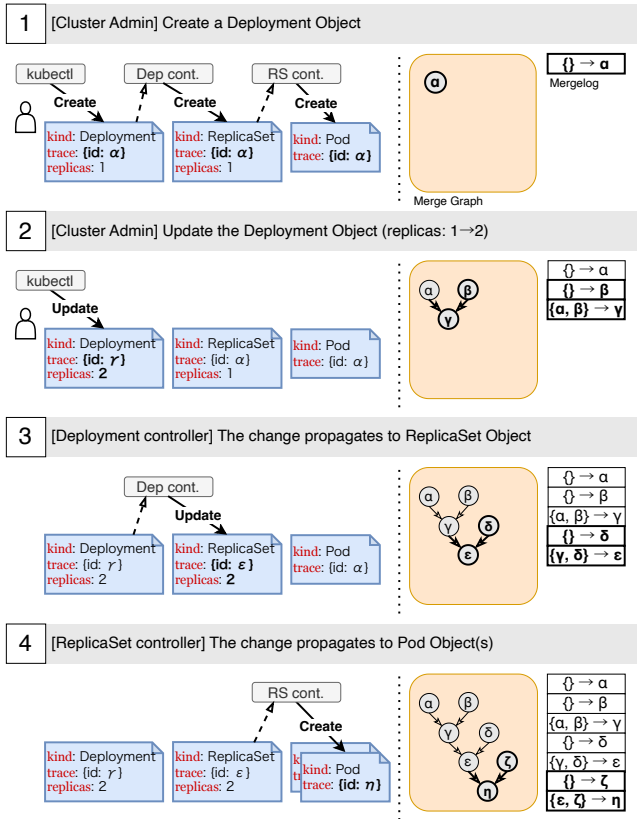


Figure 4: Situation that unnecessary merges occur.

To mitigate such inefficiencies, we add a limited number of ancestor CPIDs (CPIDs in the past) on the annotation field of each object as optional information. Suppose all objects that the controller sees to update an object (updating object) have a common

CPID in their ancestor CPIDs, and the updating object also has the same CPID. In that case, we can simply copy the CPID from the CPID in an object that the controller refer to update, because such CPID already reflect the changes pointed by the CPID in the updating object.

For example in Step 3 of Figure 4, if the Deployment object includes α in the ancestor CPIDs, the deployment controller (Dep cont.) can understand that γ includes the changes pointed by α , which is the CPID in the ReplicaSet object, and can use γ for the new CPID in the ReplicaSet object.

Since the ancestor CPIDs are optional information, they do not necessarily need to contain all of the N most recent ancestor CPIDs, and each controller may change the ancestor CPIDs unless the relationship between the CPIDs and the ancestor CPIDs is broken. This idea allows us to take advantage of temporal locality that most of processing will be completed quickly, and it helps to increase the percentage of merge process avoidance and handle change merging more efficiently. To maximize the efficiency of the ancestor CPIDs, newer CPIDs should be on the list of the ancestor CPIDs.

4 Implementation

We implemented part of the proposed system to verify that the proposed method works in practice.

4.1 Environment

We use Go 1.20 and kind 0.20.0 for implementation and measurement, and Protocol Buffers [5] and gRPC [6] for defining data format and interfaces between components. The Kubernetes cluster is created using kind [17] on a virtual machine with the specifications shown in Table 3. The controller implementation is based on the August 2023 commit¹ of the Kubernetes source code.

Table 3: Specification of host and virtual machines

	host machine	virtual machine
OS	Ubuntu20.04.3	Ubuntu20.04.6
Kernel	5.4.0	5.4.0
CPU(vCPU)	AMD EPYC 7402P	8
Memory	196GiB	24GiB

4.2 Instrumentation Library

Processes such as establishing communication with the trace server, generating CPIDs, sending mergelogs and spans, and creating child spans are common to each controller. The instrumentation library provides these operations necessary for instrumentation.

In Section 3, we stated that CPIDs must not conflict even if distributed components generate them independently. Therefore, this implementation uses UUID version 4 [9] as the CPID. UUID version 4 is also used as a Span ID.

The instrumentation library provides the following functions. The trace context here refers to a structure consisting of a CPID and a list of ancestor CPIDs.

- Inject trace contexts on a variable

¹<https://github.com/kubernetes/kubernetes/commit/02e51b27a9a40bd10094d4d87d90aff78ace171e>

- Extract trace contexts from a variable
- Inject a trace context on an object
- Extract a trace context from an object
- Generate a root CPID and it to the trace server
- Merge multiple trace contexts to generate a new CPID and send a mergelog if necessary
- Start and end a span

Algorithm 1 is an algorithm that, given multiple trace contexts (tctxs), constructs an ancestor relationship graph of CPIDs based on their CPIDs and ancestor CPIDs to determine if merging is necessary. If the graph returned by this algorithm has only one root, the CPID of that root can be determined to be the related CPID of all other CPIDs, and merge processing is unnecessary. If the graph has more than one root, it is necessary to merge those CPIDs to generate a new CPID.

Algorithm 1 Build CPID Graph

```

Input: tctxs: list of trace context
Output: cpidGraph: graph of CPID (dictionary)
1: function BUILDCPIDGRAPH(tctxs)
2:   cpidGraph ← {} // create an empty cpidGraph
3:   // for each tctx in tctxs, add it to cpidGraph
4:   for all tctx in tctxs do
5:     cpidGraph ← addCPID(cpidGraph, tctx)
6:   end for
7:   return cpidGraph
8: end function
9:
10: function ADDCPID(cpidGraph, tctxs)
11:   // Step 1: check if any of tctx.AncCpids matches to any roots.
12:   // If so, add the ancestors of the root to cpidGraph[tctx.cpid]
13:   // and delete the root from cpidGraph.roots. If not, do nothing.
14:   for all key in cpidGraph.keys do
15:     if key is in tctx.AncCpids then
16:       for all val in cpidGraph[key] do
17:         if key is not in tctx.AncCpids then
18:           // append because the end side is older ancestor
19:           add val to tctx.AncCpids
20:         end if
21:       end for
22:       delete cpidGraph[key]
23:     end if
24:   end for
25:   // Step 2: check if tctx.Cpid is included in the roots of cpidGraph.
26:   // If so, add tctx.AncCpids as values of the root. If not, add tctx to cpidGraph's
  root.
27:   cpidsIncluded ← false
28:   for all key in cpidGraph.keys do
29:     if key = tctx.Cpid || tctx.Cpid is in cpidGraph[key] then
30:       for i = len(tctx.AncCpids) - 1 to 0 do
31:         if tctx.AncCpids[i] is not in cpidGraph[key] then
32:           // prepend because the start side is newer ancestor
33:           add tctx.AncCpids[i] to front of cpidGraph[key]
34:         end if
35:       end for
36:       cpidsIncluded ← true
37:     end if
38:   end for
39:   if not cpidsIncluded then
40:     cpidGraph[tctx.Cpid] ← tctx.AncCpids
41:   end if
42:   return cpidGraph
43: end function

```

4.3 Instrumentation of Controllers

Some controllers are instrumented using the instrumentation library in Section 4.2.

kubectl kubectl is a command line tool that communicates with the API server to operate a Kubernetes cluster and, given an object definition, sends it to the API server. This time, we have implemented to generate a CPID, send a mergelog, and create an object with that CPID on it when the --trace flag is set. Also, since this is the starting point of object creation and modification, the root CPID generated by kubectl is notified to the command executor (cluster operator) so that the change propagation regarding the change can be traced.

Deployment Controller and ReplicaSet Controller When an object is changed, each controller merges the CPIDs of the objects observed in that reconciliation loop and sends a mergelog. The trace context obtained from the merge processing is placed on the object to be changed. In addition, the span is instrumented to capture the processing time for functions that are expected to take a long time. **Scheduler** The Scheduler was instrumented to generate spans by the Pod's CPID when scheduling Pod objects.

4.4 Trace Server

The trace server is implemented in Go mainly because the Kubernetes library is fully maintained. The protocol buffer compiler generates the Go language data type and communication interface code from the aforementioned scheme definitions, and the trace server is implemented using this code. In addition, the trace server is deployed as a Kubernetes Deployment object to take advantage of Kubernetes' fault tolerance and name resolution features. The structure that stores the span, mergelog list, and merge graph is implemented with exclusive control to prevent conflicts by concurrent accesses.

5 Evaluation

5.1 Overhead

We evaluate how much overhead this system incurs by measuring the time required for change propagation and the CPU and memory usage of the trace server. The pre-implementation code is used here, as described earlier in Section 4.1.

Scenario of changes The object change scenario for this measurement is as follows. The measurements were taken ten times in the two states: before (original) and after (instrumented) instrumentation. In each case, the time required to run this scenario was measured, and for the instrumented case, the CPU and memory usage of the trace server was also measured.

- (1) Create a Deployment with 10 replicas of Pods and wait for all Pods to be ready.
- (2) Change the number of Pod replicas to 20 and wait for all Pods to be ready.
- (3) Change the number of Pod replicas to 30 and wait for all Pods to be ready.
- (4) Change the number of Pod replicas to 40 and wait for all Pods to be ready.
- (5) Change the number of Pod replicas to 50 and wait for all Pods to be ready.

In this scenario, there are five change propagations that propagate from Deployment to ReplicaSet and then to Pod. The controllers that perform processing between these are kubectl, Deployment

Controller, ReplicaSet Controller, Scheduler, and kubelet, and four controllers except kubelet have been instrumented.

Results The average CPU and memory usage of the trace server during the scenario was 18 milliCPU² for CPU and 29 Mi bytes for memory, respectively. For comparison, the API server's average CPU and memory usage during the no-object-changes period was 68 milliCPU and 45 Mi bytes of memory, respectively. As can be seen by the comparison with the API server, the trace server's CPU and memory usage is insignificant and has little impact on the other workloads.

The average time required to run the scenario was 32.09 seconds before instrumentation and 73.84 seconds after instrumentation. Since five change propagations were executed, the time per propagation was 6.42 seconds before and 14.77 seconds after instrumentation. Although this time may vary depending on how much instrumentation is used (e.g., number of spanning points, number of controllers instrumented in the change propagation path), in this implementation, the time required for change propagation increased by about 8 seconds due to the instrumentation. However, considering that this method will enable debugging of delays of several minutes caused by controller bugs, increasing the time by several seconds in regular cluster operation is considered acceptable. This overhead is also expected to be reduced by optimizing the instrumentation library and instrumentation.

5.2 Effect in # of Mergelogs due to a Change in the Upper Limit N of Ancestor CPIDs

To evaluate the effect of ancestor CPIDs on mergelog reduction, we check how changing the maximum number of ancestor CPIDs N placed on an object affects the number of output mergelogs.

Scenario of changes The object change scenario for this measurement is as follows. Measurements were taken ten times each for each $N = 0, 1, 2, 3, 5, 10, 15, 20, 30$. Note that there is a fixed wait time between each step for the change to propagate to the pod at each step.

- (1) The following processes with five Deployment objects are repeated four times.
 - (a) Change the number of Pod replicas of all Deployments to 1 (total number of pods is 5)
 - (b) Change the number of Pod replicas of all Deployments to 3 (total number of pods is 15)

In this scenario, the Deployment object is updated seven times, excluding the initial creation, and for each Deployment update, the Deployment object before the update and the root CPID from the update are merged. Since each step updates the CPID of the Deployment object, the Deployment controller that observes the change performs a merge processing of the CPID of the new and old Deployment object of the corresponding ReplicaSet object. Suppose the ReplicaSet object's CPID is included in the Deployment object's ancestor CPIDs. In that case, merge processing is unnecessary, and the ReplicaSet object's CPID is simply replaced with the Deployment object's CPID. If not, the Deployment controller cannot determine that it is an ancestor, so it generates a new CPID and sends a mergelog.

²<https://kubernetes.io/docs/tasks/configure-pod-container/assign-cpu-resource>

Results Figure 5 shows the number of mergelogs sent. The horizontal axis is the upper limit of N , and the vertical axis is the number of mergelogs sent to the trace server during the scenario. The data points represent the mean, and the error bars represent the standard error.

The number of mergelogs decreases as N increases. Compared to the case where ancestor CPIDs are not used ($N = 0$), the number of mergelogs is about 25 % with $N = 5$ and 8 % with $N = 10$. It can then be read that the number of mergelogs reached the lower limit of the number of mergelogs after $N = 15$ in this number of object changes (which is seven). This is because all the ancestors' information can be placed as ancestor CPIDs without overflowing at the upper limit. This result shows that ancestor CPIDs can significantly reduce the number of mergelogs, even if the number is small.

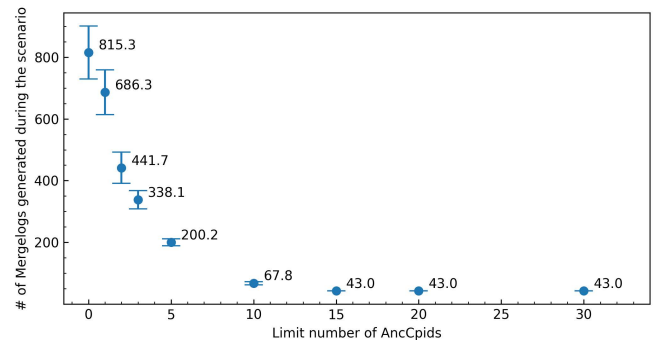


Figure 5: # of output mergelogs as the limit number of ancestor CPID (N) varies.

6 Discussion

6.1 Extensibility of the Proposed Method

Although this study targets Kubernetes, the proposed method can be extended to other systems. The behavior of the proposed method can be generalized as follows. An autonomous entity (controller in Kubernetes) reads trace contexts from something that caused the process (object in Kubernetes), embeds a trace context in the target of the change (object in Kubernetes), and outputs information (mergelog) which links those read and embedded trace contexts.

6.2 Change Propagation Traceability

This section discusses change propagation that can be traced by this method and that cannot be traced by this method. The proposed method allows change propagation to be traced by assigning CPIDs to objects. Since changes in the Kubernetes control plane are done by changing the fields of an object, in general, if change propagation is done by creating an object or changing the fields of an object, it is possible to trace the change propagation using the proposed method. However, some cases are not traceable. Examples of untraceable change propagation are discussed below.

First, some cases are inherently untraceable: changes that are propagated due to the absence of an object. Changes triggered by the deletion or non-existence of an object cannot be traced by this method if the CPID of the deleted object that caused it cannot be referenced.

Second, changes that are not represented as objects in Kubernetes cannot be traced. For example, communications and file modifications caused by an application in a container cannot be traced by this method because they are not represented as Kubernetes objects. However, these changes occur outside of the Kubernetes control plane, and because they are not represented as objects, they will not trigger other controllers, and they are outside the scope of our method. Conversely, this method can trace their changes once they are represented as objects in Kubernetes.

6.3 Reliability of the Proposed Method and Effect to the Cluster

6.3.1 System-wide Impact of Trace Server Reliability. The trace server accepts mergelogs and spans from controllers and builds the merge graph. Therefore, if the trace server is stopped, mergelogs and spans that were received during that time will be recovered. Then, tracing or retrieving spans beyond the change merges that occurred during that time will be impossible. When the trace server is down, each controller can avoid blocking its primary process by sending messages to the trace server asynchronously. One idea is to introduce retransmission logic in the controller to improve the reliability of change propagation tracing. However, since change tracing is only an auxiliary function for control plane debugging, it should not affect the performance of the original cluster systems.

6.3.2 Massive amount of mergelogs and spans. Suppose many mergelogs or spans are sent, such as when many objects are changed due to a controller bug or failure. In that case, they consume the resources of the trace server and the storage resource of the control plane and interfere with other legitimate operations. In the distributed tracing, spans sent from each service are usually sampled so that the amount handled can be kept below a certain level regardless of the amount of input. In the Kubernetes control plane, however, sampling is undesirable because if any part of the mergelog is missing, the change propagation in that part will be lost. In addition, it is impossible to sample a span for every change because no unit represents a change that always ends, such as a trace ID in the distributed tracing.

7 Conclusions and Future Work

The Kubernetes control plane, with its declarative configuration management, does not automatically trace change propagation across controllers, making it difficult to investigate the time required for change propagation and the cause of failures. In this paper, we proposed a distributed tracing method that allows change propagation to be traced by placing only one Change Propagation ID (CPID) on an object, generating a new CPID when changes merge, and placing the CPID in the regular logs issued by the span and controller. We also proposed an idea to reduce the number of mergelogs issued by placing ancestor CPIDs on the objects as auxiliary information for merging decisions. The controllers must be instrumented to propagate trace contexts in the proposed system. This makes it possible to trace any change, compared to existing systems where the types of observable resources and state transitions are fixed. It also does not require much understanding of the controller for the measurer, although the instrumentation requires some effort from its experts. We have implemented the core

functionality of the proposed method and showed that the time required for change propagation can be measured in an experimental environment. We also confirmed that the performance impact of this system on clusters is not significant. Furthermore, we showed that even a few ancestor CPIDs can significantly reduce the number of mergelogs.

Confirmation of the proposed method's scalability in larger clusters and measurement in more practical situations by implementation on third-party controllers are our future work.

Acknowledgments

This work was supported in part by JSPS KAKENHI 21K17732.

References

- [1] David Bernstein. 2014. Containers and Cloud: From LXC to Docker to Kubernetes. *IEEE Cloud Computing* 1, 3 (2014), 81–84. <https://doi.org/10.1109/MCC.2014.51>
- [2] Mike Y. Chen, Emre Kiciman, Eugene Fratkin, Armando Fox, and Eric Brewer. 2002. Pinpoint: Problem Determination in Large, Dynamic Internet Services. In *Proceedings of the 2002 International Conference on Dependable Systems and Networks (DSN '02)*. IEEE Computer Society, USA, 595–604.
- [3] Tomoyuki Ehira, Daisuke Kotani, Hiroki Shirokura, Hirofumi Ichihara, and Yasuo Okabe. 2023. Monitoring Cascading Changes of Resources in the Kubernetes Control Plane. *arXiv:2307.12567* [cs.DC]
- [4] Rodrigo Fonseca, George Porter, Randy H. Katz, and Scott Shenker. 2007. X-Trace: A Pervasive Network Tracing Framework. In *4th USENIX Symposium on Networked Systems Design and Implementation (NSDI 07)*. USENIX Association, Cambridge, MA, 271–284. <https://www.usenix.org/conference/nsdi-07/x-trace-pervasive-network-tracing-framework>
- [5] Google. 2015. *Protocol Buffers Version 3 Language Specification*. Accessed: 2024-01-29.
- [6] gRPC Authors. 2015. *gRPC - A high performance, open source universal RPC framework*. Accessed: 2024-01-29.
- [7] T. Gschwind, K. Eshghi, P.K. Garg, and K. Wurster. 2002. WebMon: A performance profiler for web transactions. In *Proceedings Fourth IEEE International Workshop on Advanced Issues of E-Commerce and Web-Based Information Systems (WECWIS 2002)*. IEEE, 171–176.
- [8] Kubernetes. 2018. *ClusterLoader*. Accessed: 2024-01-29.
- [9] Paul J. Leach, Rich Salz, and Michael H. Mealling. 2005. *A Universally Unique Identifier (UUID) URN Namespace*. <https://doi.org/10.17487/RFC4122>
- [10] Zhijian Li. 2021. *Trace context propagation*. Accessed: 2024-01-29.
- [11] Samuel Naser. 2020. *Leveraging Distributed Tracing to Understand Kubernetes Object Lifecycles*.
- [12] Abdul Qadeer. 2022. *Scaling Kubernetes to Over 4k Nodes and 200k Pods*. Accessed: 2024-01-29.
- [13] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. 2010. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Technical Report. Google, Inc. <https://research.google.com/archive/papers/dapper-2010-1.pdf>
- [14] The Cilium Authors. 2015. *Cilium*. Accessed: 2024-01-29.
- [15] The Istio Authors. 2017. *Istio*.
- [16] The Kubernetes Authors. 2014. *Kubernetes*.
- [17] The Kubernetes Authors. 2019. *kind*. Accessed: 2024-01-29.
- [18] Robert Thurlow. 2009. *RPC: Remote Procedure Call Protocol Specification Version 2*. <https://doi.org/10.17487/RFC5531>
- [19] Abhishek Verma, Luis Pedrosa, Madhukar R. Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-scale cluster management at Google with Borg. In *Proceedings of the European Conference on Computer Systems (EuroSys)*. Bordeaux, France.
- [20] VMware. 2020. *K-Bench*. Accessed: 2024-01-29.
- [21] World Wide Web Consortium. 2017. *Trace Context Level 3*. Accessed: 2024-01-29.
- [22] Zipkin. 2016. *b3-propagation*. Accessed: 2024-01-29.